Multiplatform Tetris: A Hardware Abstraction Layer Approach for Embedded and PC Systems

Sofía Margarita Vega, Juan Andrés Sanchez y Andrés Felipe Trujillo
sm_vega@javeriana.edu.co, sanchez_ja@javeriana.edu.co, af.trujillo@javeriana.edu.co

***Abstract*—**

***Keywords*—Embedded systems, hardware abstraction layer, software portability, microcontroller, ANSI C, shift register**

I. Introduction

Tetris, originally developed by Alexey Pajitnov in 1984, is among the most studied puzzle games in computing, owing to its well-defined ruleset and its suitability as a benchmark for topics ranging from artificial intelligence to software architecture [1], [2]. Its rapid expansion across multiple hardware platforms during the late 1980s and 1990s offers an early and concrete example of software portability: when game logic is separated from platform-specific input/output mechanisms, the core codebase can be reused across different environments with little modification [3].

Building on this idea, this paper presents the design and implementation of a multiplatform Tetris system written in ANSI C targeting two execution environments with very different characteristics: a personal computer terminal and an embedded system based on the Atmel ATmega328P microcontroller operating in standalone mode, without relying on the Arduino framework [4], [5]. The core challenge lies in sharing a single game engine across both platforms while adapting only the input and output layers to each environment. The system must support the full set of standard Tetris mechanics (piece generation, movement, rotation, collision detection, line clearing, and scoring) and respond to four physical inputs: move left, move right, rotate, and drop [6]. To address the portability requirement, the architecture relies on a Hardware Abstraction Layer (HAL) that places all platform-dependent operations (display rendering, input handling, and timing) behind a common interface, so that the game engine itself remains unchanged across both targets [3]. Platform selection is then handled at compile time through conditional directives, avoiding any runtime overhead.

On the PC, the game runs inside a terminal emulator and reproduces standard Tetris behavior. On the embedded side, output is shown on a logical 16×8 display made up of two cascaded 8×8 LED matrix modules [7] driven by four 74HC595 serial-to-parallel shift registers [8], [9], a setup that reduces the number of required microcontroller I/O pins through serial-to-parallel conversion and row-scan multiplexing [10]. Since the ATmega328P runs without the Arduino framework, all peripherals (UART, SPI, and the Timer) must be configured directly at the register level [5], [11], which demands a close engagement with the hardware and makes the trade-offs of bare-metal development explicit.

Throughout the project, software quality goals such as high cohesion, low coupling, consistent documentation, and platform-independent core logic are treated as central design requirements rather than secondary concerns [3], serving both technical and educational purposes.

II. Methodology

A. Matrix Control and 74HC595 Driver

The embedded display was built using two 8×8 LED matrix modules driven by four 74HC595 shift registers connected in cascade, forming a logical 16×8 panel. The 74HC595 is an 8-bit serial-to-parallel shift register with an output latch and three-state outputs, which makes it well suited for expanding the number of available output lines from a microcontroller with minimal pin usage. Each frame was stored in an 8-byte buffer, where bit j of byte i corresponded to LED (i, j). To send data to the registers, a bit-banging routine was written to toggle the SER, SRCLK, and RCLK lines directly through the ATmega328P port registers, avoiding the use of any vendor libraries. A row-scan multiplexing scheme was then implemented to activate one row at a time, holding each row for $1\,\mathrm{ms}$ and cycling through all eight rows at a global refresh rate of $125\,\mathrm{Hz}$, which was sufficient to eliminate visible flicker. On the PC side, the same buffer was rendered in the terminal by mapping each active bit to the # character, allowing early verification of the HAL before the embedded hardware was fully assembled.

B. Animation and Input Handling

C. Full Tetris Integration

III. Results

A. PC Platform Results

1) Terminal Tetris Environment: The PC implementation produced a static ASCII rendering of the Tetris environment in the terminal, as shown in Fig. 1. At this stage, consistent with the scope of the first milestone, no animation or input

handling was implemented; the result confirmed that the display abstraction layer correctly mapped the game board to character output on the desktop platform.

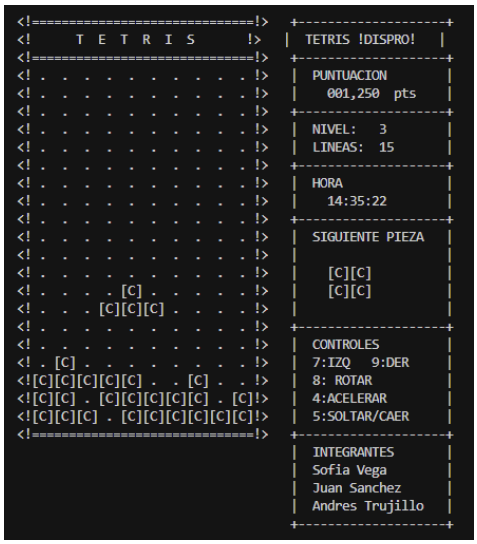

Fig. 1: Static ASCII Tetris environment rendered in the PC terminal.

B. Embedded Platform Results

1) Static Image Rendering on LED Matrices: Fig. 2 shows the result obtained on the embedded platform during the first milestone tests. Both static figures were correctly displayed on the 16×8 LED panel: a heart on the first matrix and a smiley face on the second. The images were sharp and recognizable, with no visible flicker, confirming that the 74HC595 communication and the row-scan refresh routine were operating correctly.

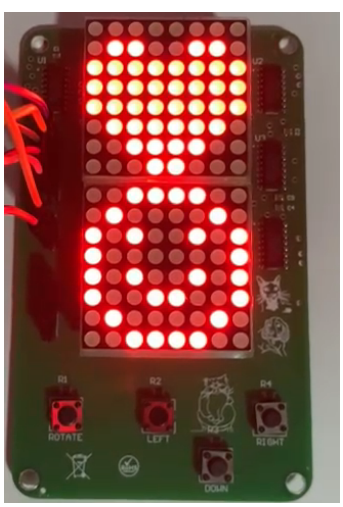

Fig. 2: Heart and smiley face rendered on the 8×8 LED matrices.

2) ASCII Verification on Serial Monitor: The same figures were also printed as ASCII art through the serial monitor, as shown in Fig. 3. This confirmed that the PC-side rendering path produced output consistent with what was observed on the physical hardware, validating the shared buffer representation across both platforms.

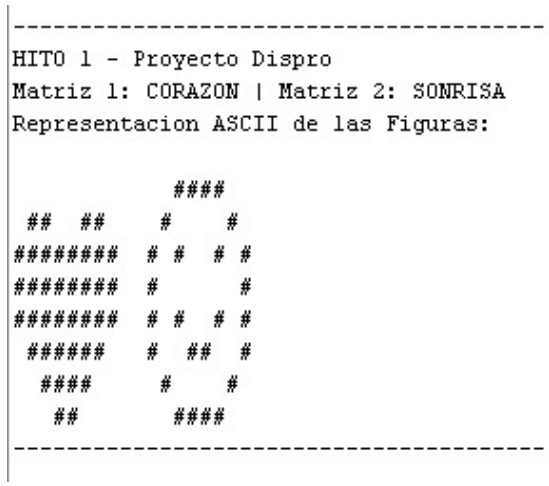

Fig. 3: ASCII representation of the heart and smiley face displayed on the serial monitor.

C. Animation and Input Handling Results

D. Full Tetris Integration Results

IV. Conclusions

References

[1] The Tetris Company. "The history of tetris," Accessed: Mar. 23, 2026. [Online]. Available: https://tetris.com/news/the-history-of-tetris.

[2] The Tetris Company. "About tetris," Accessed: Mar. 23, 2026. [Online]. Available: https://tetris.com/about.

[3] D. L. Parnas, "On the criteria to be used in decomposing systems into modules," *Communications of the ACM*, vol. 15, no. 12, pp. 1053–1058, 1972. DOI: 10.1145/361598.361623.

[4] Microchip Technology Inc., *Atmega328p: 8-bit avr microcontroller with 32k bytes in-system programmable flash*, 2015. Accessed: Mar. 23, 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/Atmel-7810-Automotive-Microcontrollers-ATmega328P_Datasheet.pdf.

[5] Microchip Technology Inc., *Avr microcontroller hardware design considerations*, AN2519. Accessed: Mar. 23, 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/AppNotes/00002519A.pdf.

[6] TE Connectivity. "Fsm2jart: Alcoswitch tactile switches," Accessed: Mar. 23, 2026. [Online]. Available: https://www.te.com/en/product-2-1437565-2.html.

[7] Kingbright Electronic Co., Ltd., *Tc23-11ywa: 58.34 mm (2.3 inch) 8x8 dot matrix display*, 2012. Accessed: Mar. 23, 2026. [Online]. Available: https://www.kingbrightusa.com/images/catalog/SPEC/TC23-11YWA.pdf.

[8] Nexperia B.V., *74hc595; 74hct595: 8-bit serial-in, serial or parallel-out shift register with output latches; 3-state*, 2024. Accessed: Mar. 23, 2026. [Online]. Available: https://assets.nexperia.com/documents/data-sheet/74HC_HCT595.pdf.

[9] Texas Instruments, *Snx4hc595 8-bit shift registers with 3-state output registers*. Accessed: Mar. 23, 2026. [Online]. Available: https://www.ti.com/lit/ds/symlink/sn74hc595.pdf.

[10] G. Leyva-Hernandez, "Led-based multiplexed display implementation using ti programmable logic devices," Texas Instruments, Tech. Rep., 2025. Accessed: Mar. 23, 2026. [Online]. Available: https://www.ti.com/lit/pdf/scla079.

[11] Microchip Technology Inc., *Avr instruction set manual*. Accessed: Mar. 23, 2026. [Online]. Available: https://ww1.microchip.com/downloads/en/DeviceDoc/AVR-InstructionSet-Manual-DS40002198.pdf.